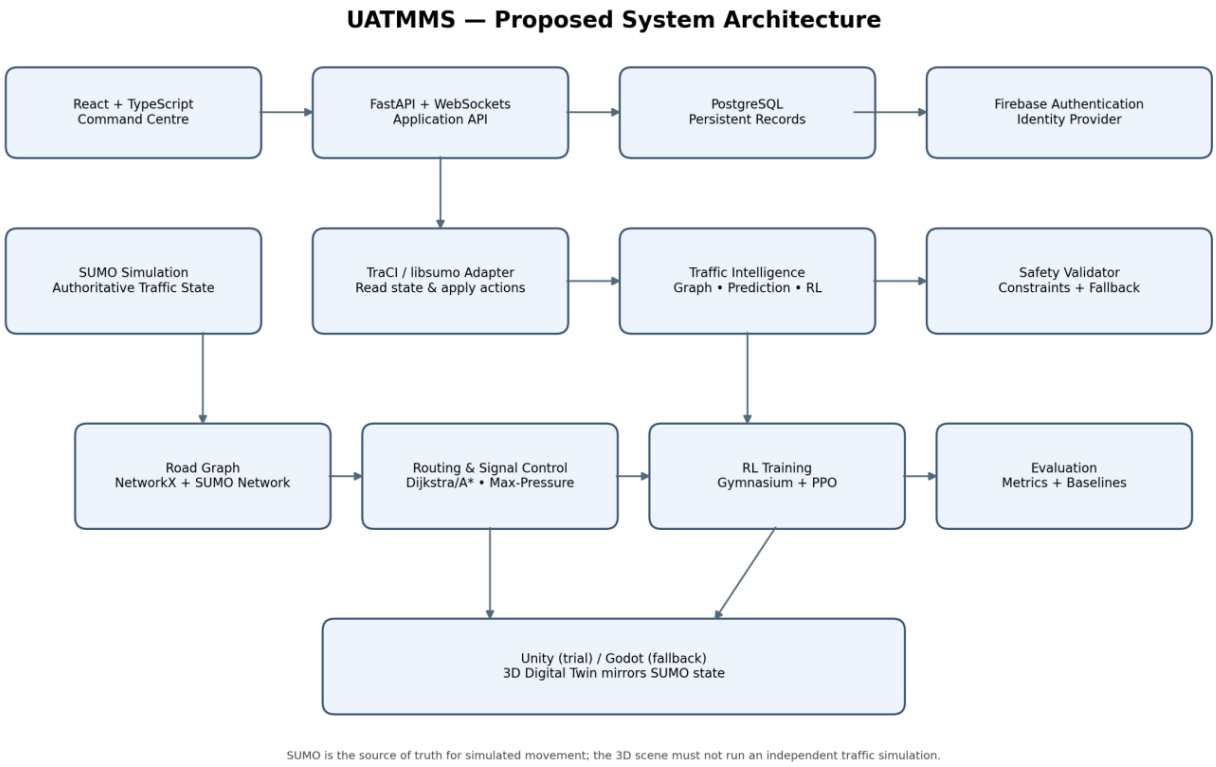
UATMMS

Universal Automatic Traffic Management and Manipulation System

Complete Project Blueprint, Technology Stack, Algorithms, Simulation Architecture and Implementation Guide



SUMO is the source of truth for simulated movement; the 3D scene must not run an independent traffic simulation.

Document purpose and how to read it

This document consolidates the project vision, prior planning discussion, proposed architecture, algorithms, technology choices, simulation strategy, role workflows, safety requirements and phased implementation plan into one shareable reference.

Status discipline
The project documents establish the intended vision and candidate technologies. This blueprint contains recommended implementation decisions. Items marked proposed, planned, optional or future are not yet implemented or experimentally validated. Do not present simulated results as real-world evidence.

Reference project documents

- Introductory_Seminar_TrafficControl_SYstem ppy fully edited(1).pdf — 14 pages.
- UATMMS_Enhanced_Project_Presentation.pptx(1).pdf — 19 pages.

Use the two project documents as primary references for the stated project vision. This blueprint adds engineering recommendations to make the vision implementable; it does not imply that all proposed features or technologies were finalized in those source documents.

Contents

- 1. Executive summary and project goals
- 2. Scope, users and operating boundaries
- 3. System architecture and simulation diagrams
- 4. Recommended technology stack
- 5. Graph model and routing algorithms
- 6. Traffic prediction and signal control
- 7. Reinforcement learning design
- 8. Emergency green corridor and incident handling
- 9. Indian-road unpredictability and scenario modelling
- 10. Data model and API plan
- 11. Role-based application and workflows
- 12. UI/3D simulation synchronization
- 13. Evaluation metrics and experiment protocol
- 14. Phased implementation plan
- 15. Testing, safety, security and risks
- 16. Repository structure and development conventions
- 17. Decisions to lock and open questions
- 18. References and implementation resources

1. Executive summary and project goals

UATMMS is envisioned as an AI-assisted, network-level traffic-management system. It observes a connected urban road network, estimates current conditions, predicts near-future congestion, coordinates route and signal decisions, supports emergency green corridors, detects incidents, and evaluates the effects of its decisions in a traffic simulator.

The core loop is: Observe → Predict → Optimize → Validate → Control → Monitor. The target demonstration is a complete browser-based command centre backed by a realistic multi-intersection simulation and, if integration succeeds, a synchronized 3D digital twin.

Primary objectives

- Monitor vehicle counts, speeds, queues, occupancy, signal phases and travel times across connected intersections.
- Represent the road network as a directed graph and compute routes using dynamic traffic costs, not distance alone.
- Predict congestion using practical machine-learning models and compare predictions against simple baselines.
- Optimize ordinary traffic while coordinating signals across a network rather than treating each road independently.
- Route ambulances and fire vehicles, schedule safe green corridors, coordinate overlapping requests and restore normal operation.
- Represent incidents and unpredictable conditions such as blocked lanes, animal crossings, rash driving and accidents in simulation.
- Introduce reinforcement learning as a candidate for learning traffic-signal policies through simulation, subject to independent safety validation.
- Compare UATMMS against fixed-time and conventional adaptive baselines using reproducible metrics.
- Provide role-based workflows and a polished dashboard whose map, graph, logs and 3D scene show the same underlying state.

Design principles

- Simulation-first; no direct control of public-road signal hardware in the initial scope.
- Network-level reasoning; avoid optimizing one road while making downstream congestion worse.
- One source of truth: SUMO owns simulated movement and signal state.
- Safety is a hard constraint, not merely a reward penalty.
- Prefer established algorithms and measurable improvements over unnecessary complexity.
- Build a functioning end-to-end slice before adding advanced AI or visual polish.

2. Scope, users and operating boundaries

In-scope first version

- Realistic multi-intersection urban network, initially a manageable subset (e.g. 6–12 intersections), later expandable.
- Simulation-generated traffic and incident data; OpenStreetMap as a source for road geometry where suitable.
- Dynamic graph routing, baseline/adaptive signal control, emergency corridor workflows, safety checks and metrics.
- Single responsive web application with role-based workspaces.
- SUMO-backed 2D network map and synchronized 3D digital-twin trial.
- Offline RL training and policy evaluation; safe deterministic fallback.

Out of scope for the initial release

- Direct connection to or control of real public-road traffic signals.
- Claiming real-world effectiveness based only on simulation.
- Fully autonomous live online RL updates without a validation and rollback process.
- Assuming CCTV computer vision, live GPS feeds or physical sensors are already available.
- Seven separate native applications; start with one role-based web app.

- Multiple databases and microservices without a measured need.

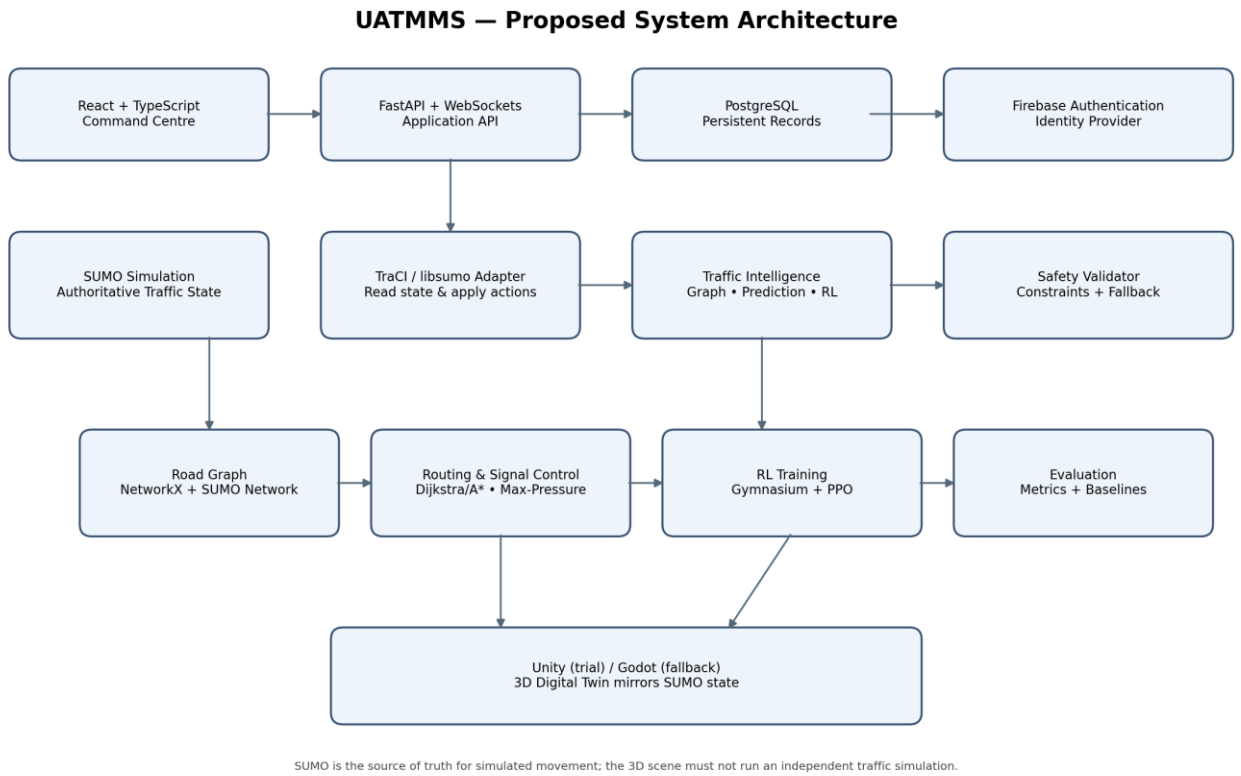
Intended roles

Role	Main capabilities (proposed)	Restrictions / notes
Admin	Manage accounts, role assignments, configuration, safety policies and analytics.	Admin access must be granted explicitly; never trust frontend role claims.
Traffic Operator	Monitor network, inspect signals, view incidents, approve permitted plans and observe system state.	Actions must pass backend authorization and safety validation.
Police	Review/verify incident reports, coordinate incident status and road closures.	Incident verification does not directly bypass signal safety.
Ambulance	Submit emergency request, view assigned route/corridor and ETA, mark status.	Only authenticated/authorized requests can trigger priority workflow.
Fire Brigade	Submit fire response request and view corridor status.	Support request prioritization and overlap handling.
Hospital	Receive emergency ETA/status and coordinate readiness.	Access only necessary emergency information.
Public / Driver	View routes, traffic/incident alerts and submit permitted reports.	No control of signal timing.

These are intended roles and permission boundaries. Final permissions should be written as a role-permission matrix before implementation.

3. System architecture and simulation diagrams

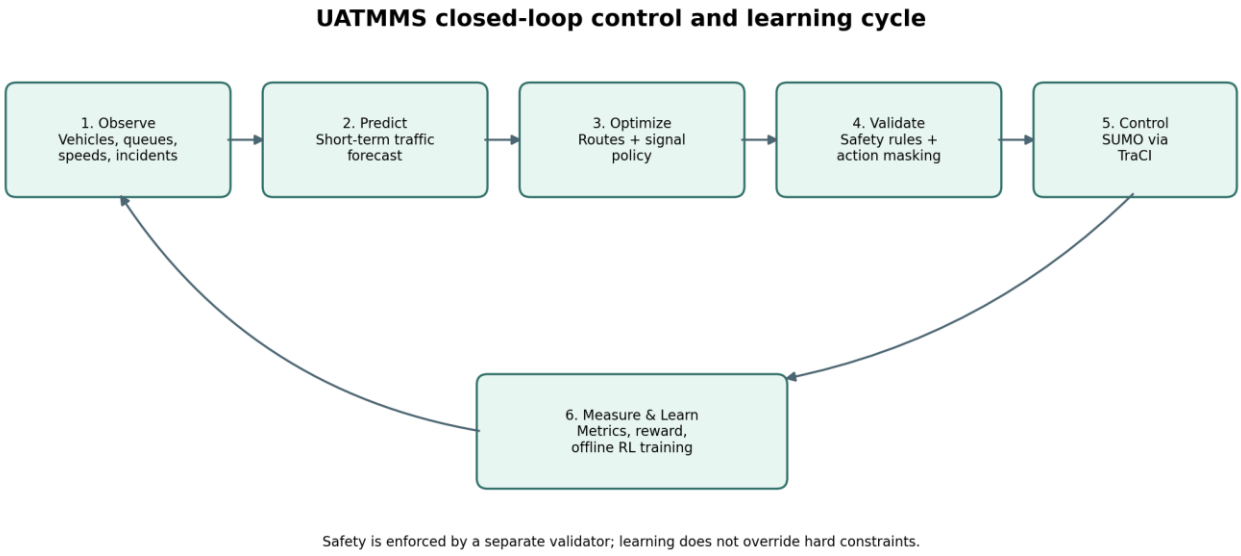
3.1 Proposed architecture



SUMO is the source of truth for simulated movement; the 3D scene must not run an independent traffic simulation.

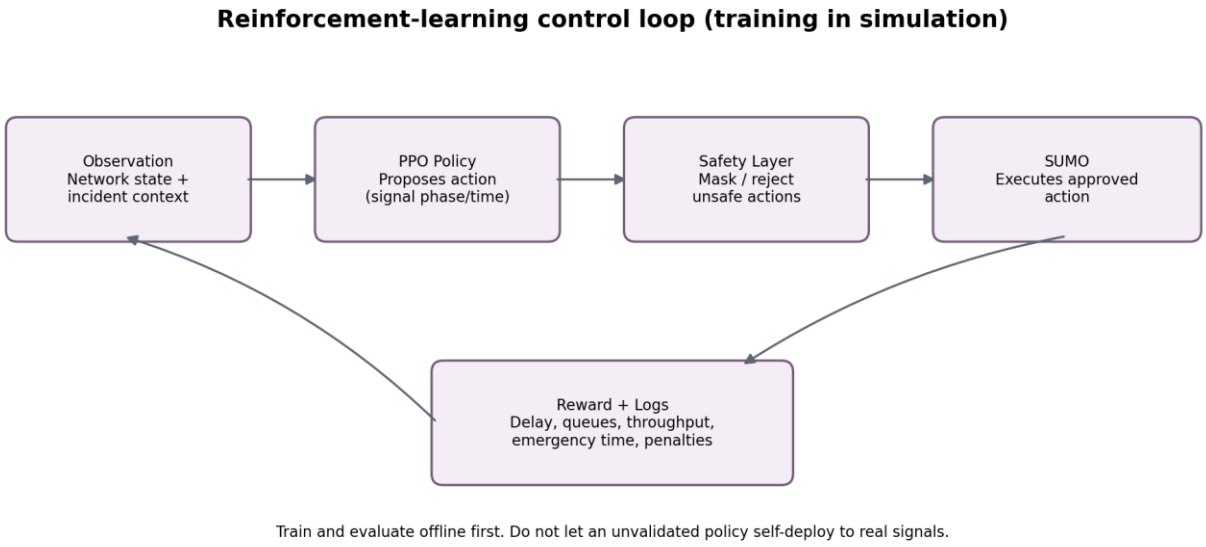
Architecture note: this is a logical diagram, not a claim that the components already exist. The initial implementation should be a modular monolith: a single FastAPI backend with well-separated Python modules.

3.2 Closed-loop traffic-management pipeline



Each cycle gathers current state, updates the road graph and predictions, proposes route/signal actions, validates them, applies approved commands through TraCI, and measures the result.

3.3 Reinforcement-learning loop



RL should initially be trained and evaluated offline in simulation. A candidate policy is versioned and promoted only after passing performance, robustness and safety checks. Online deployment to real signals is outside initial scope.

3.4 Source of truth

Component	Owns / provides	Must not do
SUMO	Vehicle movement, lane occupancy, queues, traffic-light state and simulated road evolution.	Must not be duplicated by an independent movement simulation in the 3D engine.
TraCI adapter	Reads and controls SUMO; normalizes simulation state for the backend.	Must not expose unrestricted arbitrary commands to browser clients.
Road graph	Road connectivity, directions, legal turns, dynamic travel-time weights.	Must not be mistaken for the live simulation itself.
Controller	Chooses candidate routes and signal actions.	Must not bypass the safety validator.
Safety layer	Checks hard constraints and provides deterministic	Must not rely only on a negative RL reward.

	fallback.	
Web dashboard / 3D	Visualizes shared backend/SUMO state.	Must not invent separate vehicle positions or signal states.

4. Recommended technology stack

Recommendation: use the following stack as the default unless a prototype exposes a concrete integration or performance problem. It is deliberately small and locally runnable.

Layer	Recommended choice	Why / notes	Cost / licensing note
Frontend	React + TypeScript + Vite	Single responsive role-based web app; strong ecosystem.	Open-source tooling.
UI	Tailwind CSS + shadcn/ui + Lucide	Consistent polished dashboard and components.	Check each component license; generally permissive/open source.
2D map	Leaflet + OpenStreetMap data	Network, routes, incidents and corridors.	OSM attribution/ODbL; public tile service has usage policy.
API backend	Python + FastAPI + Pydantic	Good fit for algorithms, ML and SUMO scripting.	Open source.
Live updates	FastAPI WebSockets	Push simulation state and events to UI.	No separate broker initially.
Primary database	PostgreSQL	Relational records, constraints, joins and evaluation queries.	Open source.
Flexible fields	PostgreSQL JSONB	Algorithm configuration and event payloads where schema varies.	Avoid adding MongoDB initially.
ORM/migrations	SQLAlchemy + Alembic	Models and schema migration management.	Open source.
Identity	Firebase Authentication / Google Sign-In	Managed login; verify Firebase ID tokens in backend.	Check current plan quotas, provider terms and project configuration.
Authorization	Backend role-permission checks	Role enforcement for every protected action.	Implement and test on server.
Traffic simulator	SUMO	Microscopic traffic simulation and signal control.	Open source; review current license/docs.
Simulator control	TraCI; consider libsumo if justified	Read state and apply routes/signals; libsumo can reduce overhead in compatible setups.	Open source.
Graph	NetworkX + custom domain models	Directed graph, connectivity analysis and routing baseline.	Open source.
Prediction	pandas + NumPy + scikit-learn + XGBoost	Baseline forecasts and practical ML.	Check versions/licenses; avoid deep model before baseline.
RL	PyTorch + Gymnasium + Stable-Baselines3 PPO	Simulation training and policy experiments.	Open source; validate library compatibility.
3D	Trial Unity Personal + SUMO2Unity; Godot fallback	Client-friendly 3D visualization synchronized with SUMO.	Unity proprietary and eligibility/terms apply; verify bridge and asset licenses. Godot is open source.
Charts	Apache ECharts	Live metrics, comparisons and experiment plots.	Open source.
Tests	pytest + Playwright	Algorithm/API tests and browser workflows.	Open source.
Version control	Git + GitHub	Team collaboration and issue tracking.	Git open source; GitHub service has free and paid tiers.
Deployment	Local-first; Docker/Podman later	Avoid paid cloud requirement for demo.	Check Docker licensing for intended use; Podman is an alternative.

Avoid at the start

Do not add MongoDB, Redis, Kafka, Kubernetes, microservices, Rust/Go/C++ rewrites, paid map APIs or deep RL until a measured requirement justifies them. PostgreSQL JSONB covers flexible fields; Python plus SUMO/TraCI is the shortest path to a working system.

5. Road graph model and routing algorithms

The project should reason about the entire connected network. Build a directed weighted graph G=(V,E). A node represents a junction or network connection point; an edge represents a directed road segment (and may include lane/turn metadata). Vehicles are moving entities on the graph, not graph nodes.

5.1 Graph data

Entity	Key fields (suggested)	Purpose
Intersection/node	node_id, x/y or lat/lon, signalized, junction_type, source_sumo_id	Connectivity and signal association.
Road segment/edge	edge_id, from_node, to_node, length, speed_limit, lanes, allowed_turns, direction, status	Route graph and legal movement constraints.
Dynamic edge state	edge_id, timestamp, mean_speed, vehicle_count, queue, estimated_travel_time, closure, confidence	Time-varying cost for routing.
Signal/movement	signal_id, junction_id, phase_plan, permitted_movements, min/max green, yellow/all-red constraints	Control actions and safety validation.
Vehicle/track	vehicle_id, edge_id/lane_id, position, speed, route, vehicle_type, simulated_or_detected	Current vehicle state; can be transient.
Incident/hazard	incident_id, type, location/edge, severity, start/end, status, confidence, source	Closures, obstacles and rerouting triggers.

5.2 Graph creation

1. Import or create a SUMO network; OSM is a source of road geometry, not a complete calibrated traffic-demand dataset.
2. Preserve SUMO edge, junction and traffic-light identifiers in a mapping table so controller actions target the correct objects.
3. Represent one-way streets, turn restrictions, road closures and vehicle-specific restrictions.
4. Initialize edge costs from free-flow travel time, then update them from current and predicted simulation conditions.
5. Keep the in-memory graph synchronized incrementally; avoid rebuilding the whole graph on every simulation step.

5.3 Route algorithm

Start with Dijkstra as a transparent baseline and A* as an efficient alternative where a suitable admissible heuristic is available. Use time-dependent estimated travel-time costs, not distance alone.

Example edge cost: $C_e(t)$ = estimated travel time + α × queue penalty + β × incident/risk penalty + γ × restriction penalty. The coefficients must be calibrated and evaluated; this expression is a design template, not a validated formula.

- Recalculate when congestion, incidents or closures materially change; use rerouting thresholds/hysteresis to avoid route oscillation.
- Consider downstream capacity and spillback. Independent shortest-path choices can funnel too many vehicles onto one alternative road.
- Begin with dynamic routing for selected vehicles or emergency vehicles; later evaluate network-wide traffic assignment.

5.4 Separate graphs / views

Use one physical road graph for routing and a related traffic-control graph/model describing intersections, signalized movements and upstream/downstream relationships. Share stable IDs, but keep route planning and signal optimization as separate modules.

6. Traffic prediction and signal control

6.1 Prediction

Start with simple baselines such as current-state persistence and historical/time-of-day averages. Then train XGBoost to predict future travel time, queue length or congestion class for a defined horizon (for example, 1–5 minutes depending on simulation cadence and intended task). Use held-out scenarios, not random rows from the same traffic episode, to avoid leakage.

- Inputs: current and recent speed, count, queue, occupancy, signal phase, time of day, upstream state, incident state and recent demand.
- Outputs: target-specific prediction plus timestamp and confidence/quality indicator where possible.

- Metrics: MAE/RMSE for continuous targets; precision/recall/F1 for incident classification if that task is included.
- Prediction informs routing/control but does not itself choose actions.

6.2 Signal control progression

Stage	Controller	Purpose
Baseline 1	Fixed-time signal plans	Reproducible comparison point.
Baseline 2	Queue-based adaptive control	Simple feedback controller responding to observed queues.
Candidate network controller	Max-Pressure	Uses upstream/downstream traffic pressure; practical network-level adaptive method.
Optional advanced controller	Model Predictive Control (MPC)	Evaluates candidate actions over a short horizon; adds complexity.
RL candidate	PPO policy in simulation	Learns signal actions from reward; must be compared against all baselines.

Do not claim Max-Pressure or PPO is universally best. The network, demand, action design and constraints determine results. Implement and compare controllers under identical scenarios.

7. Reinforcement-learning design

RL is proposed as an additional signal-control policy, not a replacement for the graph, route planner, prediction module or safety validator. RL learns a policy through repeated simulation; it does not inherently predict traffic or guarantee improvement.

7.1 Recommended starting point

- Algorithm: PPO via Stable-Baselines3 and PyTorch, wrapped in a Gymnasium-compatible environment.
- Environment: SUMO controlled through TraCI; begin with one intersection, then expand to a multi-intersection/shared policy.
- Observation/state: local and neighbouring queue lengths, speeds, waiting times, current phase, elapsed phase time, downstream capacity, incident/hazard flags and emergency request state.
- Actions: choose among permitted signal phases or bounded green-time extensions. Avoid arbitrary raw signal-state strings as unrestricted actions.
- Reward: combine network delay/waiting, queue/spillback, throughput and emergency travel time with penalties for excessive disruption. Hard safety constraints remain external.
- Training: randomize demand, vehicle mix, reaction times, lane changes, incidents, obstacles and emergency arrivals.
- Evaluation: compare PPO with fixed-time, queue-based adaptive and Max-Pressure on unseen seeds/scenarios.

7.2 Reward design principles

A reward may combine negative network delay, queue lengths and emergency travel time with positive throughput. Exact coefficients should be chosen through experiments. Do not use a reward penalty as the only safety mechanism, and avoid reward definitions that let the agent improve one metric while catastrophically degrading another.

7.3 Safe policy lifecycle

6. Train offline in simulation with versioned configurations and random seeds.
7. Evaluate candidate checkpoints on held-out scenarios, including severe incidents and multiple emergencies.
8. Run safety checks and compare metrics against the current policy and deterministic fallback.
9. Promote a candidate only if predeclared acceptance criteria are met; keep the previous model for rollback.
10. Monitor reward/metrics drift and out-of-distribution conditions; use fallback when observations are unreliable.
11. Do not deploy self-modifying online RL to real public-road signals within the initial project.

7.4 Multi-intersection learning

Start with one intersection to validate the environment wrapper and reward. Then try a shared policy with local plus neighbouring state. Multi-agent PPO/MAPPO or graph neural networks are future research options only after the baseline policy is stable and there is evidence that they add value.

8. Emergency green corridor and incident handling

8.1 Proposed emergency workflow

12. An authorized ambulance/fire user submits an emergency request with location, destination, vehicle type and urgency.
13. Backend validates identity, request fields and duplicate/invalid requests.
14. Routing module computes candidate network routes using dynamic travel time and legal road restrictions.
15. Corridor scheduler estimates arrival time at each junction and proposes signal changes along the route.
16. Safety validator checks phase transitions, conflicting movements, pedestrian intervals, minimum/maximum green and any competing corridor.
17. SUMO executes approved actions; dashboard and 3D view show route, signal status and ETA.
18. After passage, controller releases priority and returns signals safely to ordinary operation.
19. Record timestamps, chosen route, signal actions, travel time, red-signal encounters and any safety rejection.

8.2 Competing emergencies

- Use an explicit priority policy (severity, time waiting, location, route conflict and expected delay) approved by the project team.
- When corridors overlap, solve a conflict-aware scheduling problem; do not let requests overwrite each other's signal commands.
- Prioritize safe transition and cross-traffic risk, not simply the earliest green request.
- Evaluate one emergency, overlapping emergencies and a new emergency arriving during an active corridor.

8.3 Incident handling

The initial simulator can inject incidents directly into scenarios. A later perception module may infer incidents from camera/video data. Represent incident detection confidence and source so a simulated ground-truth event is not confused with a real detector output.

- Possible event types: blocked lane, collision/slowdown, stopped vehicle, wrong-way vehicle, animal crossing, roadwork, flooding/obstruction (if modelled), signal failure (simulation only).
- Incident response: mark affected edge/movement, adjust edge cost or closure, recompute affected routes, update traffic controller, notify dashboard, and clear incident when conditions warrant.
- Use conservative confidence thresholds and operator confirmation where automated detection could cause unsafe control.

9. Modelling unpredictable Indian-road conditions

Indian urban traffic can include heterogeneous vehicle types, varying lane discipline, informal stopping, animal crossings, sudden braking, aggressive lane changes, wrong-way movement and uncertain incident reporting. The project should model these as scenario distributions and stress tests rather than claim that a simulator automatically captures local behaviour.

Scenario family	Simulation method	What to evaluate
Mixed vehicle types	Vary passenger cars, two-wheelers, buses, trucks, auto-rickshaws and emergency vehicles where SUMO/network configuration supports them.	Travel time, throughput, queueing and controller stability.
Variable driving behaviour	Vary desired speed, following distance, reaction time, lane-	Robustness across seeds and driver-behaviour

	changing and compliance parameters.	profiles.
Animal/obstacle	Inject a moving obstacle or blocked segment; if animal dynamics cannot be represented faithfully, model a temporary lane obstruction and document the approximation.	Detection/response latency, rerouting and downstream spillback.
Accident/lane closure	Randomize location, duration, severity and capacity reduction.	Recovery time, alternate-route effects and queue spillback.
Unexpected stop / rash manoeuvre	Use supported vehicle behaviour parameters or scripted events; document approximation.	Incident response and stability of route/signal decisions.
Multiple emergencies	Randomize simultaneous origins/destinations and overlapping corridors.	Emergency travel time, conflict handling and ordinary-traffic disruption.
Sensor uncertainty	Add noise, delayed measurements or missing observations to the state estimator.	Fallback activation and degradation under uncertainty.
Demand shift	Vary arrival rates, turning ratios and peak/off-peak profiles.	Generalization to unseen demand.

Important limitation
SUMO can simulate configurable traffic behaviours and scripted incidents, but the fidelity of animal movement, Indian mixed traffic, rash driving and real perception depends on custom scenario design and calibration. State clearly when a condition is approximated by a blocked lane or scripted vehicle behaviour.

10. Data model and API plan

10.1 PostgreSQL entities (initial schema concept)

Table	Example key fields	Notes
users	id, firebase_uid, display_name, email, status, created_at	Identity is verified by Firebase; app roles live server-side.
roles / permissions	role_id, name; permission_id, key; role_permission	Use explicit role-to-permission mapping.
user_roles	user_id, role_id, assigned_by, assigned_at	Support controlled role assignment.
networks	id, name, source, config_version	One or more simulation network definitions.
intersections	id, network_id, sumo_junction_id, geometry, signalized	Stable mapping to SUMO IDs.
road_segments	id, network_id, sumo_edge_id, from_node_id, to_node_id, metadata JSONB	Keep routable topology and metadata.
scenarios	id, name, demand_profile, seed, incident_config JSONB	Reproducible scenario setup.
simulation_runs	id, scenario_id, controller_version, policy_version, start/end, status	One row per experiment/run.
incidents	id, type, location, severity, status, confidence, source, timestamps	Include simulated vs detected source.
emergency_requests	id, type, origin, destination, priority, status, requested_at	Track workflow lifecycle.
routes	id, request_id, algorithm, edge_sequence JSONB, estimated_time, actual_time	Store route result and audit trail.
signal_events	id, run_id, junction_id, old_phase, new_phase, reason, validation_result, timestamp	Audit every signal command.
metric_snapshots	id, run_id, time, avg_wait, queue, throughput, emergency_metrics JSONB	Choose sampling frequency to avoid excessive data volume.
model_versions	id, algorithm, version, artifact_uri/path, metrics JSONB, promoted_at	Version RL and prediction artifacts.
audit_events	id, user_id, action, target, result, timestamp	Security and operations audit.

The schema is a starting proposal. Use migrations and define units, timestamp conventions, ID mapping and retention policy before the team starts producing data.

10.2 API endpoints (illustrative)

Method + endpoint	Purpose
GET /api/v1/simulation/state	Current simulation time, vehicles, queues and signal state.
POST /api/v1/simulation/start	Start an authorized scenario.
POST /api/v1/simulation/pause	Pause the simulation.
POST /api/v1/simulation/reset	Reset a run and restore initial state.
GET /api/v1/network/graph	Return graph metadata for visualization.
POST /api/v1/routes/plan	Plan a route with selected cost model.
POST /api/v1/emergencies	Create an emergency request.

GET /api/v1/emergencies/{id}	Read emergency and corridor status.
POST /api/v1/incidents	Create/report an incident, with role-specific validation.
GET /api/v1/metrics/runs/{id}	Read metrics for a simulation run.
GET /api/v1/models	List model/policy versions and status.
WS /ws/simulation	Stream state/event updates to subscribed clients.

Endpoint names are illustrative. Protect every action with authentication, authorization, input validation and audit logging. Do not expose arbitrary TraCI commands directly to the browser.

11. Role-based application and dashboard workflows

11.1 Command-centre layout

- Left column: simulated camera-style views, algorithm/event logs and a switchable driver/app preview for selected users.
- Upper-right panel: network graph showing intersections, road connectivity, congestion and the controller's current understanding.
- Large central panel: 3D city/road simulation, synchronized to SUMO.
- Bottom panel: 2D map with emergency routes, traffic conditions, incidents and simulation controls.
- Alerts/side panel: active emergencies, corridor status, conflicting requests, safety rejections and system health.
- Analytics view: baseline comparisons, prediction errors, RL reward and policy versions, emergency metrics and run history.

11.2 Key workflows

Workflow	Steps
Normal traffic	Run scenario → observe network → estimate state → predict congestion → route/control → display and record metrics.
Incident	Report/detect event → validate/confirm → update edge cost/closure → reroute affected traffic → adjust signals → notify roles → track recovery.
Emergency corridor	Authorized request → route planning → arrival-time calculation → conflict/safety validation → signal schedule → vehicle passage → release/recovery → metrics.
RL training	Generate scenarios → train policy offline → evaluate on held-out scenarios → compare baselines → safety review → version candidate → controlled promotion.
Operator intervention	Operator selects permitted action → backend checks role → safety validator checks action → SUMO executes → action and outcome audited.

Camera panels in the initial demo can be rendered from simulated state or curated sample footage. They must be labelled as simulated/demo feeds until a real video pipeline is implemented.

12. UI and 3D synchronization rules

- SUMO simulation time is the authoritative simulation clock.
- Each update has a run_id, simulation timestamp and sequence/version so clients can ignore stale/out-of-order events.
- WebSockets stream summarized state; do not send every internal simulator detail at maximum frequency without profiling.
- 3D adapter maps SUMO vehicle IDs, edge/lane positions, heading and signal states into Unity/Godot objects.
- Pause, resume, reset and scenario-switch operations must update all panels consistently.
- If 3D rendering falls behind, it should interpolate display positions but never feed invented positions back into the traffic controller.
- Keep a clear mapping between road-network IDs, SUMO IDs, frontend graph IDs and 3D objects.

13. Evaluation metrics and experiment protocol

Metric	Meaning / calculation approach	Use
Average waiting time	Mean vehicle waiting time over defined run/population.	General traffic efficiency.
Average travel time	Mean completed trip time; report unfinished trips separately.	End-to-end journey performance.
Throughput	Vehicles reaching destinations or passing a defined boundary per time window.	Network capacity.
Maximum / average queue	Queue length by lane/junction and network summary.	Bottleneck and spillback.
Prediction MAE/RMSE	Error between predicted and simulated ground truth for chosen horizon.	Prediction quality.
Emergency travel time	Request-to-arrival or route travel time; define which clock is used.	Emergency response.
Red signals encountered	Count per emergency route, with definition of encounter.	Corridor effectiveness.
Safety violations	Count of invalid/conflicting signal transitions or other defined safety-rule breaches.	Must be zero for accepted test cases.
Control/API latency	Time from state snapshot to approved action / frontend update.	System responsiveness.
Recovery time	Time from incident clearance to return to defined traffic baseline.	Incident resilience.
RL reward and stability	Reward trends plus real traffic metrics; never use reward alone.	Learning diagnostics.
Ordinary-traffic disruption	Delay increase caused by emergency preemption compared with appropriate baseline.	Balance emergency and general traffic.

Fair comparison protocol

20. Use the same network, demand, turning ratios, incident placements, emergency requests and random seeds for each controller comparison.
21. Compare at least fixed-time, queue-based adaptive, Max-Pressure and the candidate RL policy.
22. Use multiple seeds and report mean plus variability; avoid selecting only the best run.
23. Separate training scenarios from evaluation scenarios; include unseen demand and incident configurations.
24. Define a warm-up period, simulation duration, vehicle population and treatment of incomplete trips.
25. Report trade-offs: a controller that improves emergency travel time but substantially worsens ordinary traffic must show both effects.
26. Predefine acceptance criteria before examining results. Do not fabricate or promise percentage improvements.

14. Phased implementation plan

Phase	Deliverable	Definition of done
0. Project setup	Repo, environment, coding conventions, issue tracker, architecture decisions.	Team can run app skeleton; dependencies and setup are documented.
1. SUMO proof of concept	One small network and Python/TraCI adapter.	Can start/pause/reset, read vehicle/queue/signal state and log data.
2. Multi-intersection network	OSM/SUMO network, graph mapping and repeatable traffic demand.	Graph IDs map correctly; simulation scenarios reproducible.
3. Routing	Dijkstra/A*, dynamic edge weights, route updates.	Rerouting responds to congestion and closures without constant oscillation.
4. Baseline signal control	Fixed-time + queue-based adaptive + Max-Pressure candidate.	Metrics generated and controller comparison repeatable.
5. Incident and emergency	Incident injection, emergency requests, corridor scheduler, safety validator.	Multi-junction emergency test passes; safe recovery is logged.
6. Prediction	Baseline forecast and XGBoost model.	Held-out prediction metrics available; prediction is not confused with control.
7. RL research branch	Gymnasium wrapper, PPO single-junction training, then shared multi-junction experiments.	Policy evaluated against baselines and unseen scenarios; safety layer remains external.
8. Backend/database/auth	FastAPI, PostgreSQL, Firebase token verification and RBAC.	Protected workflows pass tests; no frontend-only permissions.
9. Dashboard	Role-based UI, graph/map, logs, alerts and metrics.	All panels reflect shared simulation state; core workflows work.
10. 3D digital twin	Unity bridge trial or Godot integration.	Vehicle/signal state tracks SUMO through pause/reset and scenario changes.
11. Hardening/demo	Performance, tests, reproducibility, documentation and demo script.	No known critical safety/control bugs in defined test suite; limitations documented.

The team can build the backend/dashboard shell in parallel after the simulation interfaces are defined, but algorithm validation must not depend on a finished 3D interface.

15. Testing, safety, security and risks

Safety requirements

- Only configured legal signal phases and movements may be selected.
- Respect configured minimum/maximum green, yellow/all-red transitions and pedestrian intervals where represented.
- Do not issue green to conflicting movements; validate transitions against the signal plan.
- Emergency priority cannot bypass safety checks or force unsafe instant transitions.
- On controller error, stale state or low-confidence observation, move to a documented deterministic fallback/hold policy appropriate to the simulated signal state.
- Log every requested action, validator decision, applied action and rejection reason.
- Safety test cases must include overlapping emergencies, blocked routes, stale state, invalid phase requests and simulator disconnection.

Security requirements

- Verify Firebase ID tokens on the backend using the Firebase Admin SDK; never trust a client-supplied UID or role.
- Keep app role assignments in server-controlled database records; protect role changes and admin provisioning.
- Validate all API payloads; apply authorization per endpoint and WebSocket subscription.
- Do not store secrets in frontend environment variables or commit service-account credentials.
- Audit privileged actions and rate-limit sensitive endpoints where appropriate.
- Use synthetic demo accounts and simulated data; do not store real emergency/personal data unless there is an approved need.

Key project risks and mitigations

Risk	Mitigation
Too much scope / unfinished demo	Deliver simulation and routing vertical slice first; keep advanced AI optional until baseline works.
RL unstable or no better than baselines	Treat RL as experimental; use baseline controllers, reward diagnostics and held-out evaluation.
Simulation unrealistic for Indian roads	Document assumptions, randomize behaviours, calibrate when data is available and avoid real-world claims.
3D bridge incompatible	Prototype early; retain SUMO and 2D map as working demo; use custom bridge or Godot if needed.
Conflicting controller commands	Single controller orchestration point; safety validator; explicit emergency arbitration.
Database overcomplication	PostgreSQL only initially; introduce additional store only after measurement.
Authentication role escalation	Server-side authorization and automated permission tests.
OSM or model licensing issue	Track data/model/asset sources and licenses; review attribution and distribution terms.
False incident detection	Separate simulated ground truth from detected events; use confidence and operator confirmation.

16. Suggested repository structure

```
uatmms/
  apps/
    web/                        # React + TypeScript + Vite
  backend/
    app/
      main.py                   # FastAPI entry point
      api/                      # REST routes and WebSocket endpoints
      auth/                     # Firebase token verification, RBAC
      core/                     # settings, logging, errors
```

```
    db/                       # SQLAlchemy models, sessions, migrations
    simulation/               # SUMO lifecycle, TraCI adapter
    network/                  # graph import, ID mapping, edge state
    routing/                  # Dijkstra/A*, dynamic cost model
    prediction/               # baselines, XGBoost model interface
    control/                  # fixed-time/adaptive/Max-Pressure/RL adapters
    emergency/                # request lifecycle, corridor scheduler
    safety/                   # action validator and fallback
    incidents/                # event model and incident response
    evaluation/               # metrics, run manager, comparisons
    schemas/                  # Pydantic API schemas
  tests/
simulation/
  networks/                   # SUMO network files and metadata
  scenarios/                  # demand, incidents, emergency scenarios
  configs/                    # versioned run configurations
rl/
  env/                        # Gymnasium wrapper
  training/                   # PPO training scripts
  checkpoints/                # local artifacts; large files may be excluded from Git
  evaluations/                # policy benchmark results
visualization/
  unity-or-godot/             # 3D bridge and scene
docs/
  architecture.md
  safety-policy.md
  experiment-protocol.md
scripts/
  run_simulation.py
  evaluate_controllers.py
.env.example
README.md
```

Development conventions

- Use type hints and Pydantic schemas at API boundaries.
- Use explicit units and naming (meters, seconds, vehicles/minute); avoid ambiguous metric fields.
- Version scenario configs, algorithm parameters and RL checkpoints.
- Add a regression test whenever a safety or routing bug is fixed.
- Keep secrets, local database files, simulation outputs and large checkpoints out of Git unless intentionally managed.
- Use code review for signal-control, auth and safety changes.

17. Decisions to lock before implementation

Decision	Recommended default	Status
Scope	Simulation-only software; no public-road signal control.	Confirmed project constraint
Road network	Realistic multi-intersection urban network; start small, expand.	Confirmed direction
Primary database	PostgreSQL; JSONB for flexible fields.	Recommended
Backend	Python + FastAPI modular monolith.	Recommended
Authentication	Firebase Auth / Google Sign-In; server-side role checks.	Recommended
Routing	Directed weighted graph; Dijkstra baseline and A* option.	Recommended
Signal control	Fixed-time benchmark, queue-based adaptive, Max-Pressure candidate.	Recommended
Prediction	Baseline first, then XGBoost.	Recommended
RL	PPO via Stable-Baselines3/PyTorch; offline simulation training; independent safety layer.	Proposed research module
3D	Trial Unity + SUMO2Unity; Godot/custom bridge fallback.	Must validate early
Computer vision	Defer real camera detection until simulator path works; later evaluate OpenCV and a license-verified detector.	Later phase
Live data	Start with SUMO; real sensors/GPS feeds are not assumed available.	Confirmed boundary
Success criteria	Define measurable baseline and acceptance thresholds before evaluation.	Must finalize as a team

Network/demand	Choose a source city/network, scale, traffic demand and turn ratios.	Open decision
RL reward	Choose target horizon, metric weights and hard constraints through experiments.	Open decision
Role permissions	Approve exact role-permission matrix.	Open decision
Simulation cadence	Set step length, state polling interval and dashboard update rate.	Open decision

Questions the team should answer at kickoff

- What machine will run SUMO and the 3D engine, and what performance constraints does it have?
- Which urban network will be the initial demonstration network, and what size is realistic for the hardware?
- What traffic composition and behaviour assumptions will define the initial Indian-road scenarios?
- Which actions are operators allowed to perform, and which require police/administrator confirmation?
- What safety constraints are enforced in SUMO, and what is the deterministic fallback for each failure mode?
- What exact criteria must a new RL policy satisfy before it can replace the current policy in simulation?
- Will the first demo prioritize a complete software workflow or a physical hardware mock-up? The current core scope is simulation-only.

18. References and implementation resources

The following official documentation and research resources are starting points. Verify current versions, terms and integration compatibility when implementation begins.

- **SUMO documentation:** https://sumo.dlr.de/docs/ — Traffic simulation, networks, output and signal control.
- **SUMO TraCI:** https://sumo.dlr.de/docs/TraCI/ — Python interface to inspect and control simulation.
- **SUMO routing:** https://sumo.dlr.de/docs/Simulation/Routing.html — Route computation and routing options.
- **SUMO TripInfo output:** https://sumo.dlr.de/docs/Simulation/Output/TripInfo.html — Trip-level evaluation data.
- **SUMO OpenStreetMap import:** https://sumo.dlr.de/userdoc/Networks/Import/OpenStreetMap.html — Importing OSM road networks.
- **SUMO emergency simulation:** https://sumo.dlr.de/docs/Simulation/Emergency.html — Emergency-related simulation documentation.
- **SUMO2Unity repository:** https://github.com/SimuTraffX-Lab/SUMO2Unity — Potential 3D bridge; inspect compatibility, code and asset licenses.
- **CityFlow simulator:** https://github.com/cityflow-project/CityFlow — Alternative traffic-signal simulation research platform.
- **CoLight paper:** https://arxiv.org/abs/1905.05717 — Network-level cooperative traffic signal control research.
- **FHWA Signal Timing Manual, Ch. 9:** https://ops.fhwa.dot.gov/publications/fhwahop08024/chapter9.htm — Emergency preemption, transitions and recovery; US guidance, not Indian regulation.
- **FHWA Signal Timing Manual, Ch. 5:** https://ops.fhwa.dot.gov/publications/fhwahop08024/chapter5.htm — Signal timing concepts and constraints; adapt to applicable local requirements.
- **FastAPI documentation:** https://fastapi.tiangolo.com/ — API and deployment guidance.
- **PostgreSQL JSON types:** https://www.postgresql.org/docs/current/datatype-json.html — JSON/JSONB support.
- **Firebase Authentication:** https://firebase.google.com/docs/auth — Authentication integration and token verification.
- **Stable-Baselines3 PPO:** https://stable-baselines3.readthedocs.io/ — Practical RL algorithms and training API.
- **Gymnasium:** https://gymnasium.farama.org/ — Environment API for RL experiments.
- **OpenStreetMap copyright:** https://www.openstreetmap.org/copyright — Attribution and database licensing information.

Final recommendation

Build a robust simulation-backed traffic-management system first: SUMO + TraCI, a directed road graph, dynamic routing, baseline and adaptive signal control, emergency corridor safety, reproducible evaluation and a single source of truth. Add

prediction and PPO as measured enhancements, not substitutes for engineering fundamentals. Then connect the role-based React dashboard and a synchronized 3D view. Every performance claim must be backed by repeatable experiments, and every proposed feature must be labelled accurately until implemented.